

Contagem de Números Primos com OpenMP: Sequencial vs. Paralelo

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Este relatório apresenta a implementação e a avaliação experimental de um programa em C que conta a quantidade de números primos existentes no intervalo $[2, n]$, comparando uma versão sequencial com uma versão paralelizada por meio da diretiva `#pragma omp parallel for` da biblioteca OpenMP. O objetivo é observar o ganho de desempenho obtido com a paralelização, bem como identificar e discutir problemas de correção que podem surgir quando múltiplas threads acessam e modificam uma mesma variável compartilhada sem qualquer mecanismo de sincronização — no caso, a variável de contagem `contagem_par`.

2 Metodologia

O programa recebe uma lista de valores de n (1.000.000, 5.000.000, 10.000.000, 15.000.000 e 20.000.000) e, para cada um deles, executa duas versões do mesmo algoritmo de contagem de primos:

- **Versão sequencial:** percorre o intervalo $[2, n]$ em um único laço `for`, testando a primalidade de cada número com a função `verificar_primo` (teste de divisibilidade por tentativa até $\sqrt{num}$) e incrementando a variável `contagem_seq`.
- **Versão paralela:** executa exatamente o mesmo laço, porém precedido pela diretiva `#pragma omp parallel for`, que distribui as iterações entre as threads disponíveis. O incremento é feito na variável compartilhada `contagem_par`, sem uso de `reduction`, `atomic` ou `critical`.

O tempo de execução de cada versão foi medido com `omp_get_wtime()`. Os testes foram executados em uma máquina com 8 núcleos disponíveis (`omp_get_num_procs() = 8`). O código-fonte completo encontra-se no Apêndice A.

3 Resultados

A Tabela 1 resume a quantidade de primos encontrados, os tempos de execução e a diferença entre as contagens sequencial e paralela para cada valor de n testado.

A Tabela 2 apresenta o *speedup* (razão entre o tempo sequencial e o tempo paralelo), a eficiência (*speedup* dividido pelo número de núcleos) e o erro percentual da contagem paralela em relação à contagem sequencial (considerada correta).

Tabela 1: Resultados obtidos para cada valor de n (8 núcleos disponíveis).

n	Primos (Seq.)	Primos (Par.)	Tempo Seq. (s)	Tempo Par. (s)	Diferença
1.000.000	78.498	76.652	0,3710	0,1140	1.846
5.000.000	348.513	344.713	3,6930	1,0480	3.800
10.000.000	664.579	659.197	9,8250	2,8370	5.382
15.000.000	970.704	963.832	17,5090	4,8950	6.872
20.000.000	1.270.607	1.262.967	26,4810	7,4820	7.640

Tabela 2: Speedup, eficiência e erro percentual da contagem.

n	Speedup	Eficiência (%)	Erro na contagem (%)
1.000.000	3,25	40,7	2,35
5.000.000	3,52	44,1	1,09
10.000.000	3,46	43,3	0,81
15.000.000	3,58	44,7	0,71
20.000.000	3,54	44,2	0,60

A Figura 1 ilustra a evolução do tempo de execução das duas versões e do speedup obtido em função do tamanho do intervalo n .

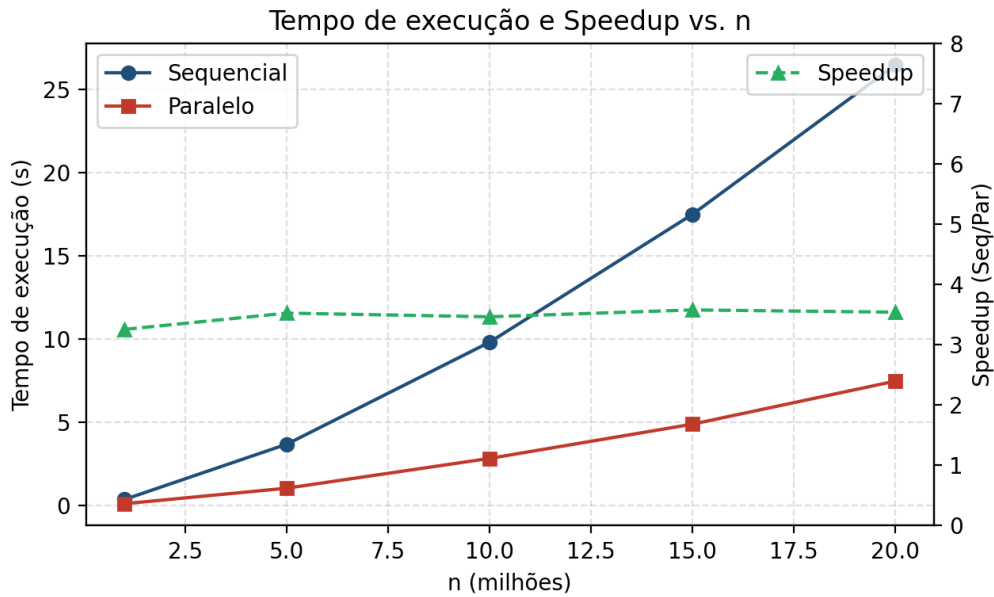


Figure 1: Tempo de execução (sequencial e paralelo) e speedup em função de n .

Em todos os cinco casos testados, a versão paralela apresentou um número de primos **menor** do que a versão sequencial, e a mensagem [ALERTA] Os resultados divergem! foi disparada em 100% das execuções.

4 Análise

4.1 Desempenho

O tempo de execução da versão sequencial cresce de forma aproximadamente proporcional a n (na verdade, um pouco mais que linear, já que o custo do teste de primalidade de cada número cresce com $\sqrt{num}$). A versão paralela acompanha essa mesma tendência de crescimento, porém com um tempo consistentemente menor. O speedup obtido variou entre $3,25\times$ e $3,58\times$, estabilizando-se em torno de $3,5\times$ para os valores de n maiores. Como a máquina possui 8 núcleos disponíveis, o speedup ideal (linear) seria próximo de $8\times$; o valor observado corresponde a uma eficiência de apenas $\sim 41\text{--}45\%$.

Esse distanciamento do speedup ideal é esperado nesta implementação simples e pode ser atribuído a:

- **Desbalanceamento de carga:** o `#pragma omp parallel for` sem uma cláusula `schedule` explícita usa, por padrão, a política `static`, dividindo o intervalo $[2, n]$ em blocos contíguos de tamanho igual entre as threads.

Como o custo de `verificar_primo(num)` cresce com $\sqrt{num}$, threads responsáveis pelas faixas de números maiores realizam mais trabalho do que as responsáveis pelas faixas iniciais, gerando desbalanceamento de carga.

- **Condição de corrida no incremento de `contagem_par`:** sem `reduction`, `atomic` ou `critical`, múltiplas threads leem, incrementam e escrevem a mesma variável compartilhada concorrentemente, o que também poder ter um efeito (secundário) na parte de custo do sincronismo implícito de cache (falso compartilhamento / *cache invalidation*), embora o principal efeito observado aqui seja sobre a *correção*, discutido a seguir.
- **Overhead de criação e sincronização das threads**, que se torna proporcionalmente menor à medida que n aumenta — o que explica por que o speedup tende a estabilizar (e até melhorar ligeiramente) para os valores maiores de n .

4.2 Correção: a condição de corrida

O resultado mais importante desta atividade não é o ganho de desempenho, mas o problema de **correção** evidenciado em *todas* as execuções: a contagem paralela foi sempre **menor** do que a sequencial, nunca maior e nunca igual.

Isso ocorre porque a operação `contagem_par++` não é atômica. Internamente, ela corresponde a três passos: (1) ler o valor atual de `contagem_par`, (2) somar 1, e (3) escrever o novo valor de volta na memória. Quando duas ou mais threads executam esses três passos de forma intercalada sobre a mesma variável, é possível que ambas leiam o mesmo valor antes que qualquer uma escreva o resultado incrementado; nesse caso, um dos incrementos é “perdido”. Como esse tipo de sobreposição depende do escalonamento das threads pelo sistema operacional, o resultado final é não determinístico — porém, com um número suficientemente grande de primos a contar (e, portanto, de incrementos disputados), a tendência estatística é sempre de *perda* de incrementos, nunca de excesso, o que explica por que a contagem paralela é sistematicamente menor.

4.3 Reflexão sobre os desafios da programação paralela

Este experimento evidencia dois desafios centrais e complementares no início da programação paralela:

1. **Correção antes de desempenho:** paralelizar um laço sem analisar as dependências entre iterações (neste caso, a dependência de todas as iterações sobre a mesma variável `contagem_par`) produz um programa mais rápido, porém incorreto. Um ganho de desempenho não tem valor se o resultado produzido não é confiável. A correção deveria ser restaurada, por exemplo, substituindo a diretiva por `#pragma omp parallel for reduction(+:contagem_par)`, que garante que cada thread mantenha uma cópia local do acumulador e that as combine de forma segura ao final, sem custo de sincronização a cada iteração.
2. **Distribuição de carga:** mesmo corrigindo a condição de corrida, o desbalanceamento causado pelo custo não uniforme de `verificar_primo` continuaria limitando o speedup máximo alcançável com um escalonamento puramente estático. Nesse caso, testar cláusulas como `schedule(dynamic)` ou `schedule(guided)` tenderia a melhorar a distribuição de trabalho entre as threads, aproximando o speedup obtido do speedup teórico permitido pelo número de núcleos.

5 Conclusão

A paralelização do laço de contagem de primos com OpenMP produziu um speedup médio de aproximadamente $3,5\times$ em uma máquina com 8 núcleos, evidenciando ganho real de desempenho, mas distante do ideal devido ao desbalanceamento de carga inerente ao escalonamento estático padrão. Mais relevante do que o ganho de tempo, porém, foi a demonstração prática de que paralelizar um laço "sem alterar a lógica original" não é suficiente para garantir resultados corretos: a ausência de sincronização no incremento de uma variável compartilhada gerou uma condição de corrida que subestimou sistematicamente a contagem de primos em todos os cinco cenários testados, com erro absoluto crescente e erro relativo decrescente à medida que n aumentava. O experimento reforça que, na programação paralela, a verificação de corretude (por meio de mecanismos como `reduction`, `atomic` ou `critical`) deve caminhar lado a lado com a busca por desempenho, e que a distribuição desigual de trabalho entre threads é um segundo aspecto que também precisa ser avaliado e ajustado (por exemplo, com diferentes políticas de `schedule`) para que o potencial de paralelismo do hardware seja de fato aproveitado.

A Código-Fonte

O código-fonte completo utilizado nos testes está disponível em `../src/tarefa5.c` e é reproduzido a seguir.

```
1  /*
2   Implemente um programa em C que conte quantos números primos existem entre 2 e um
3   valor máximo n.
4   Depois, paralelize o laço principal usando a diretiva:
5   #pragma omp parallel for
6   sem alterar a lógica original. Compare o tempo de execução e os resultados das vers
7   ões sequenciais
8   e paralela. Observe possíveis diferenças no resultado e no desempenho, e reflita
9   sobre os
10  desafios iniciais da programação paralela, como correção e distribuição de carga.
11
12  Compilação:
13  $ gcc -fopenmp tarefa5.c -o tarefa5
14  */
15
16 #include <stdio.h>
17 #include <stdbool.h>
18 #include <omp.h>
19
20 bool verificar_primo(int num) {
21     if (num < 2) return false;
22     for (int i = 2; i * i <= num; i++) {
23         if (num % i == 0) return false;
24     }
25     return true;
26 }
27
28 int main(int argc, char **argv) {
29     int valores_n[] = {1000000, 5000000, 10000000, 15000000, 20000000};
30     int qtd_valores = sizeof(valores_n) / sizeof(valores_n[0]);
31
32     printf("Núcleos disponíveis: %d\n\n", omp_get_num_procs());
33
34     for (int idx = 0; idx < qtd_valores; idx++) {
35         int n = valores_n[idx];
36         int contagem_seq = 0;
37         int contagem_par = 0;
38
39         printf("=====\n");
40         printf("Intervalo: [2, %d]\n\n", n);
41
42         // ----- VERSÃO SEQUENCIAL -----
43         double inicio_seq = omp_get_wtime();
44         for (int i = 2; i <= n; i++) {
45             if (verificar_primo(i)) {
46                 contagem_seq++;
47             }
48         }
49         double fim_seq = omp_get_wtime();
50
51         // ----- VERSÃO PARALELA -----
52         double inicio_par = omp_get_wtime();
53         #pragma omp parallel for
```

```

51     for (int i = 2; i <= n; i++) {
52         if (verificar_primo(i)) {
53             contagem_par++; // Condição de corrida ?
54         }
55     }
56     double fim_par = omp_get_wtime();
57
58     printf("Sequencial : %d primos encontrados | tempo: %.4f s\n",
59           contagem_seq, fim_seq - inicio_seq);
60
61     printf("Paralelo : %d primos encontrados | tempo: %.4f s\n",
62           contagem_par, fim_par - inicio_par);
63
64     if (contagem_seq != contagem_par) {
65         printf("\n[ALERTA] Os resultados divergem! Diferenca = %d\n",
66               contagem_seq - contagem_par);
67     } else {
68         printf("\nOs resultados coincidiram nesta execucao (pode nao ocorrer sempre
69               ).\n");
70     }
71     printf("\n");
72 }
73
74 return 0;
75 }

```

Código 1: Contagem de primos sequencial e paralela (OpenMP).